

Arquitectura del sistema Deasy

Guia de lectura completa del repositorio

PUCE Sede Esmeraldas

9 de agosto de 2026

Sobre este documento. Es una explicación detallada y didáctica de la arquitectura del monorepo DEASY, escrita asumiendo cero contexto previo. Recorre el negocio, las ocho piezas que corren en contenedores, el camino completo de una petición HTTP, el backend capa por capa, el modelo de datos y el motor de procesos, el frontend, el microservicio de firma, la infraestructura y el despliegue, la estrategia de pruebas, las trampas conocidas del repositorio y una ruta de lectura sugerida.

Todo lo que se afirma aquí procede de la lectura directa del código: rutas de fichero, nombres de tabla, números de línea y conteos son reales en el momento de escribirlo.

Índice general

1. Que es Deasy (el problema de negocio)	1
2. Vista de pajar: ocho piezas en contenedores	2
3. El recorrido completo de una peticion	4
4. El backend en detalle	5
4.1. Las capas y la regla no negociable del proyecto	5
4.1.1. Que hay dentro de cada capa	5
4.1.2. Inventario de routers	6
4.2. El punto de entrada: <code>backend/index.js</code>	6
4.3. El acceso a datos: un adaptador peculiar	7
4.3.1. <code>bindParams</code> : traduce los marcadores de posicion	7
4.3.2. <code>translateDialect</code> : traduce funciones	7
4.3.3. Transacciones al estilo mysql2	7
4.3.4. Donde vive el SQL	8
4.4. El CRUD generico de administracion	8
4.5. Autenticacion y autorizacion	9
4.5.1. Autenticacion (JWT)	9
4.5.2. Autorizacion (RBAC)	9
4.6. Manejo de errores	10
4.7. Integraciones externas	11
4.7.1. RabbitMQ, con sorpresa	11
4.7.2. MinIO: siete buckets	11
4.7.3. Socket.IO (tiempo real)	11
4.7.4. Correo, WhatsApp y servicios ecuatorianos	12
4.8. Los scripts del backend	12
5. La base de datos y el motor de procesos	13
5.1. Vocabulario del dominio	13
5.2. El motor de procesos: serie, regla y flujo	14
5.2.1. Tabla por tabla	14
5.3. <code>item_mode</code> : los tres modos de emision	15
5.4. Plantillas: el modelo “libro y ediciones”	16
5.4.1. El contenido: Jinja2 sobre LaTeX	16
5.4.2. Ciclo de vida y versionado	16
5.5. Firmas	16
5.5.1. El flujo de firma en lote	17
5.6. El dossier (ex-MongoDB)	17
5.7. Los demas dominios de tablas	18
5.7.1. Identidad y personas	18

5.7.2.	Organizacion, unidades y puestos	18
5.7.3.	Entregables, tareas y documentos	18
5.7.4.	Chat y notificaciones	18
5.7.5.	Auditoria	18
6.	El frontend	19
6.1.	Organizacion: modular por dominio	19
6.2.	El arranque	19
6.3.	Estado: no hay Pinia ni Vuex	19
6.4.	La capa HTTP	20
6.5.	El router y sus guards	20
6.6.	Composables destacados	21
6.7.	Tiempo real en el cliente	21
6.8.	Vue Flow y dagre: los dos grafos	22
6.9.	Los “God Objects” (deuda tecnica visible)	22
7.	El microservicio signer	24
7.1.	Arquitectura interna	24
7.2.	El flujo completo de una firma, paso a paso	24
7.3.	sigmaker/: por que hay Node dentro de un servicio Python	25
7.4.	trust/: la cadena de confianza	25
7.5.	Los OIDs ecuatorianos: 500 lineas de <code>app.py</code>	26
7.6.	Pruebas del signer	26
8.	Infraestructura y despliegue	27
8.1.	Docker: base mas overlays	27
8.1.1.	Diferencias por entorno	27
8.1.2.	Los servicios del fichero base	28
8.2.	<code>scripts/docker-env.sh</code> : la interfaz comun	28
8.2.1.	Los demas scripts operativos	28
8.3.	nginx: dos configuraciones distintas	29
8.3.1.	Mecanica de plantillas	29
8.3.2.	Desarrollo	29
8.3.3.	Produccion y QA	29
8.3.4.	Rutas efectivas	29
8.3.5.	Certificados y ACME	29
8.4.	CI/CD	30
8.4.1.	<code>cd-multienv.yml</code> : ocho jobs	30
8.4.2.	Dos modos de entrega	30
8.4.3.	<code>sonar.yml</code> : solo manual, y en verde si falta configuracion	30
8.5.	SonarQube: como esta montado	31
9.	Testing: dos niveles que no se mezclan	32
9.1.	Reglas de ubicacion	32
9.2.	Los tests unitarios	33
9.3.	Los characterization tests (golden master)	33
9.3.1.	Anatomia del harness	33
9.3.2.	El ciclo de trabajo	33
9.4.	Comandos de validacion por modulo	34
9.5.	Las reglas al mover codigo	34
10.	Cosas que te van a confundir (y no son bugs)	35
10.1.	Usuarios y accesos de referencia	36

11. Por donde empezar a leer	37
11.1. Los tres documentos maestros	37
11.2. Las tres cosas que mas cuestan	37

Capítulo 1

Que es Deasy (el problema de negocio)

Antes de la arquitectura, el **para que**. Deasy es una plataforma de la **PUCE Sede Esmeraldas** para gestionar los **tramites documentales recurrentes** de la universidad: informes de gestion docente, requerimientos, informes de investigacion formativa, memorandos, etc.

Hoy, sin Deasy, esos tramites viven en Word y PDF por correo: nadie sabe quien debe entregar que, en que periodo, ni quien falta por firmar. Deasy modela cuatro cosas:

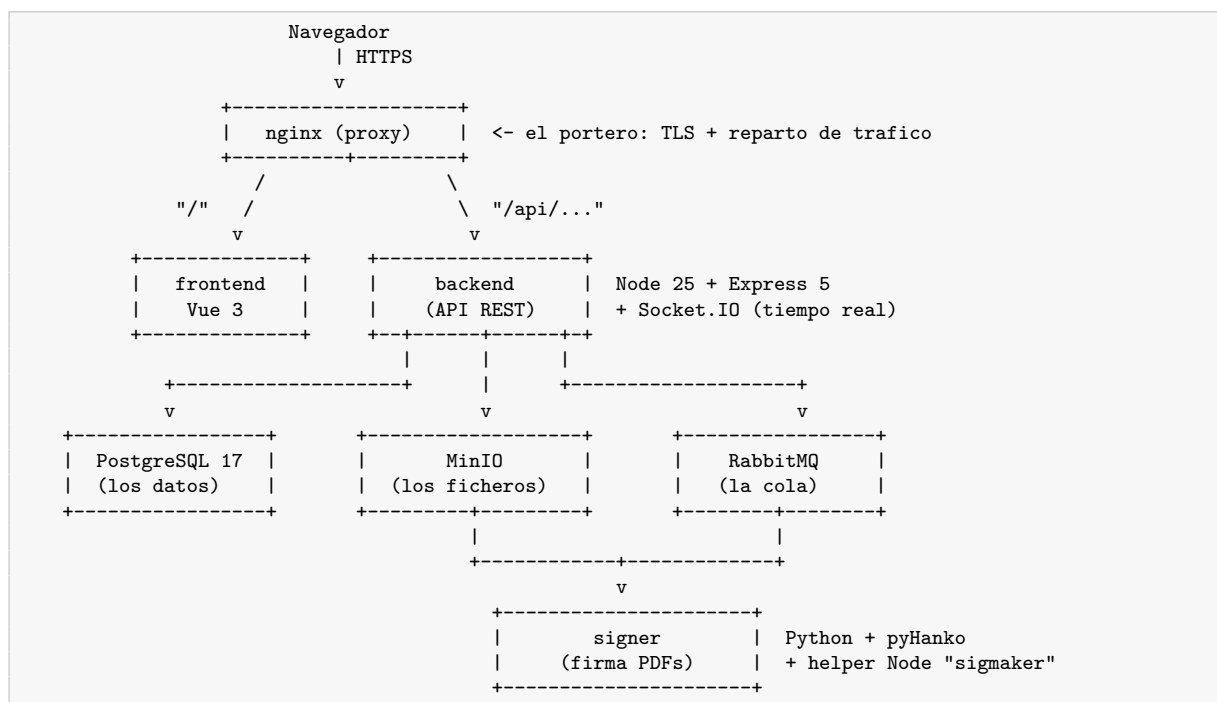
Pregunta	Como la modela
¿Quien?	Organigrama: unidades → puestos → personas ocupandolos
¿Que?	Plantillas de documento, versionadas
¿Cuando?	Periodos academicos (terms)
¿Con que recorrido?	Flujos de <i>entrega</i> (quien llena y revisa) y de <i>firma</i> (quien firma y en que orden)

Y encima de todo eso, **firma electronica real** (PAdES, con certificados .p12 de las autoridades certificadoras ecuatorianas: Security Data, ICERT y Firmasegura).

Capítulo 2

Vista de pajarito: ocho piezas en contenedores

Todo el sistema esta **dockerizado**. No hay un “servidor” monolitico, sino ocho procesos que se hablan por red.



Traducción de cada pieza para alguien que empieza:

- **nginx:** el “portero” del edificio. Es lo unico expuesto a internet. Termina el HTTPS (descifra) y, segun la URL, manda la peticion al frontend o al backend. Tambien es lo que permite que el frontend llame a `/api/...` sin problemas de CORS.
- **frontend:** la aplicacion web (Vue 3). En produccion es HTML, JS y CSS estatico servido por un nginx pequenito dentro de su propia imagen.
- **backend:** la API. Recibe peticiones HTTP, decide que se puede hacer, guarda en PostgreSQL, sube ficheros a MinIO y encola trabajos de firma en RabbitMQ.
- **PostgreSQL:** la base de datos relacional. **Es el unico motor de datos:** MariaDB y MongoDB fueron retirados. Veras comentarios “ex-MongoDB” en el codigo; son cicatrices de esa

migracion.

- **MinIO**: almacenamiento de objetos compatible con S3. Los PDFs, plantillas, fotos de perfil y certificados **no** se guardan en la base de datos ni en el disco del backend: van aqui, en *buckets* (contenedores de primer nivel).
- **RabbitMQ**: una cola de mensajes. Sirve para que el backend le pida al signer “firma este PDF” **sin quedarse bloqueado** esperando.
- **signer**: microservicio Python que hace la firma criptografica real.
- **analytics**: hoy es un marcador de posicion (CMD `["sleep","infinity"]`), no hace nada.

Por que esa separacion importa. Si la firma se hiciera dentro del backend, un PDF de 200 paginas bloquearia el servidor para todos los usuarios mientras se procesa. Al ponerlo detras de una cola, el backend delega el trabajo pesado y sigue atendiendo peticiones normalmente.

Capítulo 4

El backend en detalle

Es la pieza mas grande: unas 30.000 lineas de JavaScript **ESM** (import y export, no require), Express 5, sin ORM.

4.1. Las capas y la regla no negociable del proyecto

routes/	"que URL?"	Solo define paths y encadena middlewares
v		
controllers/	"transporte"	Valida entrada, llama a UN servicio, responde HTTP
v		
services/	"la logica"	Reglas de negocio, transacciones, maquinas de estado
v		
config/postgres.js		Acceso a datos (pool de conexiones)

El fichero CLAUDE.md del repositorio lo dice sin rodeos: “**Los controllers son transporte, no logica**”. Si un controller pasa de unas 40 lineas o abre una transaccion, hay que extraer un servicio. Esto no siempre se ha respetado — hay infractores historicos como `user_controler.js` con 1.513 lineas — y estan catalogados como deuda tecnica en `docs/planes/referencia/calidad-y-medicion.md`.

4.1.1. Que hay dentro de cada capa

- **routes/**: 14 ficheros planos (`user_router.js`, `sign_router.js`, `admin_router.js`, `chat_router.js`, `dossier_router.js`, etc.). Todos cuelgan del prefijo `/deasy/v1`.
- **controllers/**: 8 subcarpetas por dominio (`users/`, `admin/`, `sign/`, `chat/`, `tareas/`, `system/`, `whatsapp/`, `empresa/`), 27 ficheros en total.
- **services/**: 18 subcarpetas, unos 66 ficheros. Aqui estan los dominios reales: `admin/` (con sub-subcarpetas `kernel/`, `crud/`, `templates/`, `processes/`, `org/`, `generation/`), `auth/`, `chat/`, `documents/`, `sign/`, `system/`, `tasks/`, `users/`, `mail/`, `storage/`, `realtime/`, `infrastructure/`, `whatsapp/`, `external/`.
- **database/**: solo dos ficheros — `postgres_initializer.js` y `postgres_schema.sql`. No es una capa de repositorios, es el arranque del esquema.

Un detalle de diseno elegante en `services/admin/`: la carpeta `kernel/` **no importa a nadie** y todos importan de ella. Nada apunta “hacia arriba”. Eso evita dependencias circulares, que en JavaScript producen errores dificilissimos de diagnosticar en tiempo de ejecucion.

4.1.2. Inventario de routers

Router	Montaje	Lineas	Autenticacion
user_router.js	/users	318	mixta (login y create son publicos)
admin_router.js	/admin	32	authMiddleware + loadAccessContext a nivel de router
sql_admin_router.js	/admin/sql	139	hereda del padre + requireSqlAdminPermission
sign_router.js	/sign	93	por ruta
dossier_router.js	/dossier	85	authMiddleware + loadAccessContext
chat_router.js	/chat	53	authMiddleware (sin RBAC)
whatsapp_router.js	/whatsapp	25	ninguna
tarea_router.js	/tarea	15	solo en /supervised-stuck
reset_password_router.js	/reset-password	14	publica
notification_router.js	/notifications	12	authMiddleware
email_router.js	/email	9	publica
program_router.js	/program	9	ninguna
unit_router.js	/units	9	ninguna
system_router.js	/system	9	publica (bootstrap)

Duplicaciones reales. `program_router.js` y `unit_router.js` son *identicos*: ambos exponen `getPrograms` y `createProgram` del mismo controlador. Y `notification_router.js` duplica dos endpoints que ya estan en `chat_router.js`.

4.2. El punto de entrada: backend/index.js

Monta los middlewares en este orden exacto:

#	Que monta	Detalle
0	<code>import "dotenv/config"</code>	primera linea del fichero
1	<code>app.set("trust proxy", 1)</code>	va detras de nginx
2	<code>app.use(cors(...))</code>	envuelto en un middleware propio; loguea el origen en <i>cada</i> peticion
3	<code>app.use(express.json())</code>	parser JSON global
4	<code>app.use(cookieParser())</code>	necesario para la cookie <code>refreshToken</code>
5	<code>app.get("/health")</code>	llama a <code>assertPostgresConnection()</code> ; 200 o 503
6	Swagger UI	en <code>/deasy/docs</code>
7	<code>/deasy/docs.json</code>	el JSON crudo del spec
8	Los 14 routers	ver tabla anterior
9	<code>express.static("public")</code>	la carpeta <code>backend/public/</code> esta vacia

El arranque (`startServer()`) hace, en secuencia:

```
startServer()
|- initializeDatabaseWithRetry() // bucle de reintentos
|   |- assertPostgresConnection() // config/postgres.js
|   +- ensurePostgresSchema({reset}) // database/postgres_initializer.js
|- http.createServer(app)
|- realtimeGateway.init(httpServer, { corsOrigin, credentials: true })
|- httpServer.listen(PORT) // PORT || 3030
+- IIFE async: SqlAdminService().reconcileArtifactWorkflows({ onlyStale: true })
```

Los reintentos se parametrizan con `DB_INIT_MAX_ATTEMPTS` (20), `DB_INIT_RETRY_DELAY_MS` (3000) y `DB_RESET_SCHEMA_ON_START`. Si se agotan los reintentos, **el backend no aborta**: loguea el fallo y sigue escuchando.

Que NO se conecta al arrancar. RabbitMQ no abre conexion (se usa su API HTTP bajo demanda). MinIO construye el cliente al importar el modulo, pero no crea buckets al arrancar. El bot de WhatsApp solo se inicializa por peticion explicita. Y Swagger se configura con `apis: []`, o sea que **no** escanea anotaciones JSDoc: todo el spec es un objeto estatico en `backend/config/swagger/definition.js` (581 lineas).

4.3. El acceso a datos: un adaptador peculiar

Aqui hay algo que confunde si no se explica. El fichero `backend/config/postgres.js` (512 lineas) **no es un cliente de PostgreSQL normal**. Es un **adaptador que finge ser mysql2/promise**.

¿Por que? El proyecto **migro de MariaDB a PostgreSQL**. Habia unos 881 sitios en el codigo escribiendo SQL en estilo MySQL. Reescribirlos todos era inviable, asi que se escribio una capa que traduce al vuelo. Hace tres cosas.

4.3.1. `bindParams`: traduce los marcadores de posicion

MySQL usa `?`; PostgreSQL usa `$1`, `$2`:

```
// Lo que escribe el programador:
pool.query("SELECT * FROM persons WHERE cedula = ? AND status = ?", [ced, "Activo"])

// Lo que llega a PostgreSQL:
"SELECT * FROM persons WHERE cedula = $1 AND status = $2"
```

Y es cuidadoso: no toca los `?` que estan dentro de comentarios (`-`, `/* */`) o de literales de texto, porque mantiene una tabla de “tramos protegidos”. Ademas expande parametros de tipo array igual que hacia `mysql2`: un escalar da `$n`; un array vacio da `NULL`; `[1,2,3]` da `$1`, `$2`, `$3`; y `[[1,2],[3,4]]` da `($1,$2)`, `($3,$4)`.

Un cambio reciente e importante: **falla ruidosamente si faltan parametros**. Antes mandaba `NULL` en silencio, que es peor porque el fallo aparece lejos de su causa. El mensaje de error, a proposito, *no* incluye el SQL: varios controllers responden `error.message` directamente al cliente.

4.3.2. `translateDialect`: traduce funciones

`IFNULL` → `COALESCE`; `GROUP_CONCAT` → `string_agg`; `CURDATE()` y `CURTIME()` → sus equivalentes; `FROM DUAL` desaparece; `<=>` → `IS NOT DISTINCT FROM`; `INSERT IGNORE` → `ON CONFLICT DO NOTHING`; `IF()` → `CASE WHEN`; `FIELD()` → un `CASE` posicional; `YEAR()`, `MONTH()` → `EXTRACT`.

El caso mas delicado es `ON DUPLICATE KEY UPDATE` → `ON CONFLICT (...) DO UPDATE SET`, porque PostgreSQL exige que le digas *sobre que columnas* es el conflicto. El adaptador lo **infiere consultando el catalogo** de PostgreSQL (`pg_index`) y cachea el resultado en memoria.

4.3.3. Transacciones al estilo `mysql2`

```
const conn = await pool.getConnection();
await conn.beginTransaction(); // ejecuta BEGIN
try {
  await conn.query("INSERT ...");
  await conn.query("UPDATE ...");
  await conn.commit();
} catch (e) {
  await conn.rollback();
}
```

```

} finally {
  conn.release();           // devuelve la conexion al pool
}

```

Hay 11 ficheros de produccion que usan transacciones, entre ellos `services/admin/generation/launch.js`, `services/admin/templates/templateLifecycle.js`, `services/documents/FillRequestWorkflowService`, `services/sign/PdfSigningService.js` y `services/system/SystemBootstrapService.js`.

Que es un “pool” de conexiones. Abrir una conexion a la base de datos es caro (decenas de milisegundos: handshake de red, autentificacion, negociacion). Un *pool* mantiene N conexiones ya abiertas y te presta una cuando la necesitas. Aqui `max = 10`. Por eso es **crítico** llamar siempre a `release()`: si te olvidas, se agotan las diez y la aplicacion entera se cuelga esperando una conexion que nunca vuelve.

El SQL no lo valida nadie hasta que se ejecuta esa rama. Textual del CLAUDE.md. El SQL es una cadena de texto: `node -check` no la mira, el detector de imports tampoco, y el backend arranca igual. Asi sobrevivieron meses *cuatro* sentencias `UPDATE ... INNER JOIN ... SET` (sintaxis multi-tabla de MySQL que PostgreSQL rechaza) que dejaron el endpoint `POST /sign/fill-requests/:id/return` roto para todo el mundo.

PostgreSQL quiere `UPDATE tabla alias SET col = ... FROM otra WHERE union AND filtros`, con las columnas del SET **sin cualificar**. Y `grep "UPDATE.*JOIN"` *no encuentra nada*, porque el SQL ocupa varias lineas.

4.3.4. Donde vive el SQL

`getPostgresPool` se importa en **28 ficheros**: 17 en `services/`, 5 en `controllers/`, y el resto en `config/`, `database/` y `utils/`. Es decir, **no hay una capa de repositorios unica** — es una deuda conocida. Solo `services/auth/` tiene repositorios nominales (`UserRepository.js`, `UserCertificateRepository.js`).

4.4. El CRUD generico de administracion

Esta es una de las piezas mas ingeniosas del backend. En vez de escribir un controlador para cada una de las 46 tablas administrables, hay tres ficheros que colaboran:

- `backend/config/sqlTables.js` (1.009 lineas): un **catalogo declarativo** de metadatos. Para cada tabla describe sus campos, etiquetas humanas, tipos, cuales son obligatorios y cuales de solo lectura.
- `backend/services/admin/SqlAdminService.js` (884 lineas): lee ese catalogo y **construye** los `SELECT`, `INSERT`, `UPDATE` y `DELETE` dinamicamente.
- `backend/services/admin/crud/tableHooks.js` (1.137 lineas): los “injertos” — la logica especial que necesita cada tabla concreta.

Una entrada del catalogo tiene esta forma:

```

{
  table: "unit_types",
  label: "Tipos de unidad",
  category: "Estructura",
  primaryKeys: ["id"],

```

```

fields: [
  { name: "name", label: "Nombre", type: "text", required: true },
  ...
],
searchFields: ["name"]
}

```

Ventaja: anadir una tabla al panel de administracion es anadir una entrada al catalogo, no escribir un CRUD entero. **Desventaja:** `tableHooks.js` se ha convertido en un cajon de sastre. El `CLAUDE.md` avisa: “*No injertes casos especiales en el camino generico. Es el olor que hizo God a AdminTableManager*”.

4.5. Autenticacion y autorizacion

Son **dos cosas distintas** y conviene no confundirlas nunca:

- **Autenticacion** = “¿quien eres?” → JWT
- **Autorizacion** = “¿puedes hacer esto?” → RBAC

4.5.1. Autenticacion (JWT)

```

POST /users/login { cedula, password }
|
v
AuthService.login() -> bcrypt.compare(password, password_hash)
|
v
Devuelve DOS tokens:
* access token -> 2 horas -> en el JSON; el frontend lo guarda en localStorage
* refresh token -> 30 dias -> en una COOKIE httpOnly (el JS no puede leerla)

```

Por que dos tokens. El *access token* viaja en cada peticion, asi que si te lo roban el dano dura poco (dos horas). El *refresh token* solo se usa para pedir uno nuevo, y al ser una cookie `httpOnly`, un script malicioso inyectado en la pagina **no puede leerlo** desde JavaScript. Es defensa en profundidad: dos secretos con exposiciones distintas.

Detalles concretos: el access token se firma con `JWT_SECRET` y expira en `60*60*2` segundos; el refresh con `JWT_REFRESH` y expira en `60*60*24*30`. La cookie lleva `httpOnly: true` y `secure` activo salvo en modo desarrollador.

El payload del JWT es **solo** `{ uid }`. Nada de roles ni permisos dentro. Eso es deliberado: si metieras los permisos en el token, revocarle un rol a alguien no tendria efecto hasta que expirase. Aqui los permisos se resuelven **contra la base de datos en cada peticion**.

La politica de contraseñas vive en `backend/utils/passwordPolicy.js` (`evaluatePasswordPolicy`, exige 3 de 5 criterios) y la aplica `backend/middlewares/val_password.js`, que ademas *hashea en el sitio* `req.body.password` con `bcrypt` y salt 10 antes de llamar a `next()`.

4.5.2. Autorizacion (RBAC)

RBAC son las siglas de *Role-Based Access Control*, control de acceso basado en roles. El modelo es:

```

resources x actions --> permissions ("dossier.read", "signature_flows.update", ...)
|
v

```

```

role_permissions --> roles
                    |
                    v
                role_assignments (persona + rol + UNIDAD)
                    ^ derived_from_assignment_id
                position_assignments <-- cargo_role_map

```

Trece recursos (account, dossier, security, people, units, academic_terms, process_definitions, process_execution, templates, documents, fill_flows, signature_flows, contracts) por cinco acciones (read, create, update, delete, manage) dan **65 permisos**. Y hay **13 roles**.

Dos sutilezas importantes:

1. **Los roles son contextuales a una unidad.** Puedes ser **GestorProcesos** en la Facultad de Ingenieria y nadie en el resto de la universidad. La tabla `role_assignments` lleva `unit_id` y un `max_depth`: hasta que profundidad del subarbol organizativo se hereda el rol.
2. **Hay roles derivados automaticamente del puesto.** La tabla `cargo_role_map` dice, por ejemplo, “el cargo **DOCENTE** otorga el rol **GestorEjecucionProcesos**”. Y un **trigger de PostgreSQL** (`trg_position_assignments_after_insert`) crea esas asignaciones con `source='derived'` cuando alguien ocupa el puesto, y las revoca cuando lo deja. Ese mismo trigger **reasigna los entregables abiertos** al nuevo ocupante.

Los middlewares estan en `backend/middlewares/rbac.js`:

Middleware	Que comprueba
<code>loadAccessContext</code>	Carga roles y permisos en <code>req.auth / req.access</code> ; 401 si el usuario no existe o esta inactivo
<code>requirePermissions(reqs, {all})</code>	Que tenga el permiso (OR por defecto, AND con <code>{all:true}</code>)
<code>requireAnyRole(roles)</code>	Que tenga alguno de los roles indicados
<code>requireRouteUserAccess({...})</code>	Que sea el dueno del recurso o tenga rol elevado, y ademas el permiso
<code>requireCedulaAccess({...})</code>	Igual, pero comparando por cedula
<code>requireDossierAccess(action)</code>	Azucar sintactico sobre el anterior con <code>resource: "dossier"</code>
<code>requireSqlAdminPermission(...)</code>	Deduca el recurso desde <code>req.params.table</code> y la accion desde el metodo HTTP

Por que existe `requireDossierAccess`. Por un **IDOR** real y cerrado. IDOR (*Insecure Direct Object Reference*) es el fallo donde cambiando un identificador en la URL accedes a datos que no son tuyos. Aqui el guard miraba *la tarea* en vez de *el entregable*, y un docente podia descargar el documento de otro.

El catalogo canonico esta en `backend/config/rbacCatalog.js` (229 lineas) y es tambien la **fuentes de siembra**: `SystemBootstrapService.js` lo importa para poblar roles y permisos en la base de datos, borrando y reescribiendo `role_permissions` en cada arranque. O sea que el **catalogo del codigo es la fuente de verdad**, no la base de datos.

4.6. Manejo de errores

Hay una decision de diseno explicita: **no hay un middleware de error global**. Cada controller hace su propio `catch`:

```

catch (error) {
  res.status(error.statusCode ?? 500).json({ message: error.message });
}

```

`backend/errors/HttpError.js` define la clase y las fabricas: `notFound()` (404), `forbidden()` (403), `conflict()` (409), `badRequest()` (400). El comentario de cabecera del fichero dice literalmente que “*un error SIN statusCode sigue siendo un 500 — y eso esta bien*”.

Aparte, `backend/errors/sqlErrors.js` traduce los codigos SQLSTATE de PostgreSQL a mensajes humanos: 23505 (violacion de UNIQUE) da “ya existe una persona con esa cedula”. Y lo hace comparando por **nombre de constraint** (`error.constraint`), no buscando subcadenas en el mensaje, que es fragil y se rompe al cambiar de idioma o de version.

Un cuarto elemento: `backend/middlewares/uploadError.js` traduce los siete codigos de error de `multer` a mensajes en espanol y **oculta el mensaje si es un 5xx**. Se monta con `router.use(handleUploadError)` *despues* de las rutas, en tres routers (`user_router`, `dossier_router`, `sign_router`). Existe porque sin el, Express devolvía HTML con el stack trace completo, incluidas las rutas absolutas dentro del contenedor.

4.7. Integraciones externas

4.7.1. RabbitMQ, con sorpresa

Aunque `amqplib` figura en `package.json`, **no se usa en ningun sitio**. Toda la comunicacion va por la **API HTTP de gestion** de RabbitMQ (`backend/services/infrastructure/rabbitmq_http.js`). El patron es **RPC con polling**:

```
Backend genera correlationId = randomUUID()
-> crea la cola de respuesta deasy.sign.response.<uuid>
-> publica el trabajo en deasy.sign.request
-> hace polling cada SIGN_POLL_MS (1000 ms) de la cola de respuesta
    hasta SIGN_TIMEOUT_MS (120000 ms)
```

Las tres colas: `deasy.sign.request`, `deasy.sign.validate.request` y el prefijo `deasy.sign.response`. No hay consumidores persistentes en el backend: el consumidor real es el microservicio signer.

4.7.2. MinIO: siete buckets

Bucket	Variable de entorno	Contenido
<code>deasy-templates</code>	<code>MINIO_TEMPLATES_BUCKET</code>	plantillas y semillas
<code>deasy-documents</code>	<code>MINIO_DOCUMENTS_BUCKET</code>	documentos generados
<code>deasy-dossier</code>	<code>MINIO_DOSSIER_BUCKET</code>	respaldos del CV personal
<code>deasy-spool</code>	<code>MINIO_SPOOL_BUCKET</code>	area temporal de firma
<code>deasy-chat</code>	<code>MINIO_CHAT_BUCKET</code>	adjuntos del chat
<code>deasy-users</code>	<code>MINIO_USERS_BUCKET</code>	fotos de perfil
<code>deasy-certificates</code>	<code>MINIO_CERTIFICATES_BUCKET</code>	certificados .p12

La lista canonica esta en `backend/scripts/lib/reset_targets.mjs`.

4.7.3. Socket.IO (tiempo real)

`services/realtime/RealtimeGateway.js` (179 lineas) esta montado sobre el **mismo servidor HTTP** que Express, en el path `/socket.io`. Sustituyo a un broker MQTT (EMQX).

- **Autenticacion:** con el **mismo JWT** que la API, leído de `socket.handshake.auth.token`. Al cliente siempre se le dice “Token invalido”; el motivo real va al log.
- **Rooms** (mapeo uno a uno de los antiguos topics MQTT): `user:{personId}` (join automatico), `conversation:{id}`, `process:{id}`.
- **Eventos entrantes:** `conversation:subscribe`, `conversation:unsubscribe`, `process:subscribe`,

`process:unsubscribe`, todos con callback de confirmacion. La suscripcion **reutiliza la autorizacion REST**, no la reimplementa.

- **Eventos salientes:** `chat.message.created` y `chat.notification.created`.

4.7.4. Correo, WhatsApp y servicios ecuatorianos

`backend/lib/mailer.js` son once lineas de `nodemailer` con el host SMTP **escrito a fuego** en el codigo. El bot de WhatsApp (`services/whatsapp/WhatsAppBot.js`, 272 lineas) usa `whatsapp-web.js` sobre Puppeteer headless y muestra el QR de vinculacion por terminal; se inicializa manualmente y sus seis endpoints **no llevan autentificacion**. Y `services/external/webservices_ec.js` valida cedulas y numeros ecuatorianos contra un servicio externo.

4.8. Los scripts del backend

Fichero	L.	Para que
<code>reset.mjs</code>	114	CLI de reset con objetivos positivos y explicitos : <code>node scripts/reset.mjs <db storage> [-yes]</code> . Sin argumentos imprime el uso y sale sin tocar nada . Antes eran tres ficheros con borrado por defecto y opt-out (<code>-keep-db</code>); se invirtio la logica.
<code>lib/reset_targets.mjs</code>	155	Libreria: <code>resetPostgres()</code> , <code>resetMinio()</code> (vacía los 7 buckets) y <code>describeContents()</code> (inventario previo al borrado).
<code>bootstrap_admin_recovery.mjs</code>	46	Recupera o recrea el administrador. <code>npm run recover:admin</code> .
<code>check_missing_imports.mjs</code>	124	Detector de “símbolo movido pero no importado” . Ver el aviso de abajo.
<code>seed_dev_rich.mjs</code>	284	Fixture rica de desarrollo: crea un segundo proceso en modo single y lo lanza, porque con el proceso unico del bootstrap la pantalla <code>/home</code> no se puede verificar.
<code>generate_demo_certificates.mjs</code>	220	Genera certificados PKCS#12 autofirmados con <code>openssl</code> y los sube a <code>deasy-certificates</code> , porque la interfaz solo permite <i>subir</i> un <code>.p12</code> .

`npm run check:imports` es obligatorio tras mover codigo. `node -check` valida **sintaxis, no imports**. Un símbolo movido sin su `import` es sintaxis perfectamente valida, el modulo **carga sin quejarse**, y revienta en tiempo de **llamada**. Así estuvieron rotos tres semanas cuatro `ReferenceError` en produccion (`createUnitWithParent`, `getCargoCodeMap`, `getNextStorageVersionForTemplateCode`, `loadTemplateArtifactMetaDocument`).

Comprobar que el backend arranca **no sustituye** a ejecutar este script. Esta en CI.

No hay migraciones incrementales. No existe nada tipo Flyway o Knex. El esquema se aplica **entero** en cada arranque desde `backend/database/postgres_schema.sql`, apoyandose en que es idempotente (`CREATE TABLE IF NOT EXISTS`, `CREATE OR REPLACE FUNCTION`, siembras con `ON CONFLICT`).

Capítulo 5

La base de datos y el motor de procesos

67 tablas y una vista, en un unico fichero: `backend/database/postgres_schema.sql` (1.398 lineas).

Consecuencia de no tener migraciones. El esquema entero se reaplica en cada arranque. Es simple y funciona bien sin datos de produccion, pero significa que **no puedes hacer un ALTER TABLE evolutivo** de forma comoda: cualquier cambio de columna hay que pensarlo como idempotente o pasa por un reset.

5.1. Vocabulario del dominio

Termino	Que es en Deasy	Donde vive
Proceso	El tramite institucional en abstracto (“Informe de Gestion Docente”). Es solo un nombre y una jerarquia; el comportamiento esta en sus <i>configuraciones</i> .	<code>processes</code>
Configuracion	La version vigente del proceso: a quien alcanza, en que periodos corre, que entregables produce. Se activa y se retira.	<code>process_definition_versions</code>
Serie	El <i>eje</i> por el que un proceso se declina: por tipo de unidad, por cargo, o <code>default</code> (sin variacion).	<code>process_definition_series</code>
Regla	Quien recibe el proceso: que unidades, que cargo o puesto, con que politica de reparto.	<code>process_target_rules</code>
Corrida (run)	El acto de <i>lanzar</i> la configuracion en un periodo. Genera las tareas.	<code>process_runs</code>
Tarea	La instancia del proceso para un ambito concreto (una unidad, un periodo). Es el contenedor.	<code>tasks</code>
Entregable	Cada documento concreto a producir dentro de la tarea, con responsable, vencimiento y estado.	<code>task_items</code>
Plantilla	El <i>molde</i> del documento: schema de campos + cuerpo (Jinja2/LaTeX u ofimatico) + formatos, versionado y almacenado en MinIO.	<code>deliverables + template_artifacts</code>
Flujo de entrega	Cadena de pasos “quien llena y aprueba el documento antes de firmarlo”.	<code>fill_flow_* / fill_requests</code>
Firma	Firma electronica PAdES sobre el PDF, con certificado .p12 del firmante.	<code>signature_flow_* / document_signatures</code>
Dossier	El expediente o CV personal (titulos, experiencia, publicaciones). <i>No</i> es el expediente de un proceso.	<code>dossiers + dossier_items</code>

5.2. El motor de procesos: serie, regla y flujo

Esta es la **parte central del dominio** y la que mas cuesta al principio. La frase mnemotecnica del proyecto:

*la **serie** nombra el proceso, la **regla** reparte el proceso, el **flujo** reparte los pasos.*

```
processes                                "Informe de Gestion Docente" (solo un nombre)
|
+--> process_definition_versions          LA CONFIGURACION (draft / active / retired)
|    ^ series_id -> process_definition_series  <- LA SERIE (el eje de variacion)
|
+--> process_target_rules                 <- LA REGLA: a que unidades y cargos alcanza
+--> process_definition_period_types       <- en que tipos de periodo corre
+--> process_definition_templates         <- que documentos produce (+ item_mode)
|
+--> process_runs                         <- EL LANZAMIENTO en un periodo concreto
|
+--> tasks                               <- una por (definicion x periodo x unidad)
|
+--> task_items                           <- LOS ENTREGABLES concretos
|
+--> documents -> document_versions
|
+--> document_fill_flows / fill_requests  (entrega)
+--> signature_flow_instances / signature_requests (firma)
```

5.2.1. Tabla por tabla

processes. Solo identidad: name, slug unico, parent_id autorreferencial, is_active. **Sin comportamiento.** Se hizo DROP de unit_id, program_id, person_id y term_id por vestigiales.

process_definition_series — la serie. source_type puede ser unit_type, cargo o default. Es el **eje de variacion**: el mismo proceso puede tener una configuracion distinta por tipo de unidad o por cargo. La funcion buildProcessDefinitionVersionName genera el nombre resultante: “<Proceso> por <Serie>”, salvo en default, que usa el nombre del proceso a secas.

process_definition_versions — la configuracion. Tiene status (draft, active, retired) y vigencia (effective_from / effective_to). Dos mecanismos de integridad:

- Una **columna generada** active_series_flag mas un indice UNIQUE sobre (process_id, variation_key, active_series_flag) garantiza a **nivel de base de datos** que solo hay una version activa por linea.
- Un **trigger** (trg_process_definition_versions_before_update) impide activarla si no tiene al menos una regla activa y un tipo de periodo, lanzando una excepcion con mensaje en espanol.

Reglas de negocio dentro de la base de datos. Es poco habitual y merece la pena entender el porque: si la regla “no se puede activar sin reglas de alcance” viviera solo en el codigo, cualquier script, migracion o consulta manual podria saltarsela. Al ponerla en un trigger, **es imposible dejar la base de datos en un estado invalido**, venga la escritura de donde venga.

process_target_rules — la regla. Reparte el alcance con tres piezas: unit_scope_type (unit_exact, unit_subtree, unit_type, all_units), el cargo_id o position_id dentro de la unidad, y una recipient_policy (all_matches, one_per_unit, exact_position). Ademas priority y vigencia.

process_definition_templates — el paquete de entregables. Vincula configuracion con plantilla: `process_definition_id + template_artifact_id`, mas `sort_order` y, sobre todo, `item_mode`.

process_runs — la corrida. `run_mode` (automatic o manual), `source_run_id` autorreferencial para relanzamientos, `created_by_user_id`, `reason` y `status`. Relanzar es **una corrida nueva**, no sobrescribir la anterior.

tasks. Con un UNIQUE sobre (`process_definition_id`, `term_id`, `normalized_scope_unit_id`) — donde la ultima es una columna generada `COALESCE(scope_unit_id, 0)` — que garantiza **idempotencia del lanzamiento**: lanzar dos veces no duplica tareas.

5.3. item_mode: los tres modos de emision

Vive en `process_definition_templates.item_mode`, es decir, **en el vinculo plantilla-proceso**, no en la plantilla. Es un matiz importante. Define *cuando* nace el entregable y *de donde sale su flujo*:

Modo	Flujo	Instanciacion	Ejemplo
single	Predefinido en la plantilla (entrega y firma)	Una instancia automatica al lanzar, por cada responsable que resuelvan las reglas	Informe de Investigacion Formativa
replicated	Predefinido	Sin fan-out automatico : el responsable crea N replicas etiquetadas que heredan el flujo del original	Un requerimiento por cada docente nuevo
routed	Ninguno	El usuario define entrega y firma al instanciar (en tiempo de ejecucion)	Tareas ad-hoc

Truco de implementacion para distinguirlos en la base de datos: los flujos *de plantilla* tienen `task_item_id IS NULL`; los flujos creados en tiempo de ejecucion tienen `task_item_id != NULL`. Los materializa `materializeRuntimeFlowForTaskItem`.

Las replicas son `task_items` con `origin_kind='user_added'` enlazadas por `source_task_item_id` al original.

El “Proceso por defecto” (slug `default`, sembrado por el bootstrap) es un `routed` comodin para tareas ad-hoc que no pertenecen a ningun proceso (por ejemplo, “haz el informe de este evento”). El CLAUDE.md insiste: **no es “memorandums”**.

Resolutores de flujo autorables. Solo `task_assignee` (“Responsable del entregable”) y `cargo_in_scope` (“Por cargo”) en plantillas *official*; `specific_person` se anade en *ad_hoc*. Estan **deprecados en la autoria web** `document_owner`, `position` y `manual_pick`: siguen en el ENUM por legado.

document_owner NO esta retirado: sigue vivo. Es el resolver **mas usado** de la base de dev. La fase P1.4 retiro el atajo del *proceso por defecto* (`SystemBootstrapService.js`), pero dejo el de `BASE_META_YAML` — un `meta.yaml` escrito a mano como literal de codigo unas 250 lineas mas arriba, **en el mismo fichero**. Y ademas **se auto-replica**: `createTemplateArtifactVersion` copia los objetos de MinIO en binario, asi que cada version nueva lo hereda.

No lo des por muerto sin consultar fill_flow_steps. El plan para retirarlo es el frente 0 de `docs/planes/plan-maestro-2026-08.md`.

5.4. Plantillas: el modelo “libro y ediciones”

```

template_seeds      (semillas del catalogo: source_path, preview_path, seed_type)
  |
  v
deliverables        EL LIBRO    -> code UNIQUE, display_name, owner_process_id,
                                template_scope (official | ad_hoc), owner_person_id
  | 1:N
  v
template_artifacts  LA EDICION  -> storage_version, lifecycle_state, base_object_prefix,
                                available_formats, schema_object_key, content_hash,
                                parent_version_id (autorreferencial = linaje)

```

Con UNIQUE sobre (deliverable_id, storage_version). El esquema lleva comentarios explícitos: la identidad, el proceso propietario, el scope, la semilla y la persona propietaria viven en deliverables; template_artifacts guarda **solo** el estado y el almacenamiento de cada version.

La palabra “entregable” significa dos cosas. En sqlTables.js la tabla task_items se etiqueta “Entregables” (la *instancia* a producir). En el esquema, deliverables es “el libro”: la *identidad de la plantilla*. Es una fuente real de confusion al leer el código; siempre hay que mirar el contexto.

5.4.1. El contenido: Jinja2 sobre LaTeX

El cuerpo del documento es un **contrato Jinja2 + LaTeX** empaquetado en MinIO, no en la base de datos. El bootstrap publica la semilla backend/services/system/seeds/informe-general con schema.json, meta.yaml, data.yaml, main.tex.j2 y make.sh, y valida el pipeline: render Jinja2 con StrictUndefined → pdflatex → PDF.

La **propiedad** se expresa por el prefijo dentro de MinIO, en base_object_prefix: System/... para las plantillas curadas por el administrador, Users/{cedula}/... para las subidas por un gestor. El pipeline de maduración documentado es:

office (Word/Excel/PDF subido por el gestor) → jinja2 (curado por el administrador)

Deuda declarada. Falta *cablear* el render Jinja2 a PDF en tiempo de ejecución: hoy render_seed.py es utilería de línea de comandos, y el backend no lo invoca.

5.4.2. Ciclo de vida y versionado

template_artifacts.lifecycle_state admite draft, published y retired (por defecto published). El versionado es **por linaje**: una fila nueva con storage_version nuevo y parent_version_id apuntando a la anterior. templateLifecycle.js retira la versión publicada previa del mismo código al publicar, y valida que haya al menos un paso de entrega. Ese último requisito **se relaja para routed**, que por definición no autoriza flujo.

5.5. Firmas

```

signature_flow_templates -> signature_flow_steps -> signature_flow_instances
                                                    |
                                                    v
signature_requests -> document_signatures

```

`signature_flow_steps` soporta **multiples firmantes por paso** (columna `signers` de tipo JSONB) con quorum configurable via `approval_mode`:

- **and**: firman **todos** los del paso.
- **or**: basta **cualquiera**.
- **at_least**: un minimo de N, indicado en `required_signers_min`.

La columna `anchor_refs` (JSONB) enlaza el paso con anclas visuales dentro del PDF, es decir, donde se dibuja la firma.

Los catalogos de estado se siembran en el propio esquema: `signature_statuses` (firmado, fallido, invalido, cancelado) y `signature_request_statuses` (pendiente, en_progreso, completado, rechazado, cancelado).

5.5.1. El flujo de firma en lote

Endpoint	Funcion
POST <code>/sign/</code>	Firma individual
POST <code>/sign/validate</code>	Validacion de un PDF ya firmado
POST <code>/sign/batch</code>	Retirado — devuelve error y redirige a <code>/batch/start</code>
POST <code>/sign/batch/start</code>	Arranca el trabajo asincrono (hasta 30 PDFs)
GET <code>/sign/batch/:jobId</code>	Estado del trabajo
GET <code>/sign/batch/:jobId/download</code>	Descarga el ZIP con los resultados

El estado se persiste en `signature_batch_jobs` (clave `job_id`, mas `status`, `total`, `processed`, `success_count`, `failed_count` y `results` JSONB), con un comentario revelador en el esquema: *“persistido para sobrevivir reinicios del backend (antes vivia en un Map en memoria)”*.

Separacion de responsabilidades: `BatchSigningService.js` gestiona el ciclo del trabajo y el bucle en segundo plano, pero **no** decide donde se guarda el PDF ni habla con el firmante — eso es de `PdfSigningService.js`, que llama documento a documento. El ZIP se empaqueta con `/usr/bin/zip` (ruta absoluta a proposito).

Ademas del PDF, cada documento del lote lleva metadatos de contexto (`signatureRequestId`, `documentVersionId`, `processName`, `unitLabel`, `termName`, `stepName`), que es lo que permite persistir la evidencia del flujo de trabajo y no limitarse a firmar ficheros sueltos.

5.6. El dossier (ex-MongoDB)

El dossier es el CV o expediente personal: titulos, experiencia, publicaciones. Antes vivia en MongoDB; ahora son **dos tablas**:

- **dossiers**: la raiz, UNIQUE por `person_id`, con la `cedula` desnormalizada e indexada.
- **dossier_items**: `dossier_id` (con borrado en cascada) + `section` + **data JSONB** + `url_documento`.

Se conservo la flexibilidad de documento usando JSONB. La justificacion esta en el comentario del esquema: *“documento heterogeneo (CV con secciones) accedido siempre como arbol completo por cedula”*.

Las antiguas “colecciones” de Mongo son ahora valores de `section`: `titulos`, `experiencia`, `referencias`, `formacion`, `certificaciones`, `articulos`, `libros`, `ponencias`, `tesis`, `proyectos`.

Por compatibilidad, los `_id` se exponen como **String** para preservar el contrato que tenia Mongo, y los valores por defecto de cada seccion replican exactamente los del antiguo esquema de Mongoose.

5.7. Los demas dominios de tablas

5.7.1. Identidad y personas

persons (con cedula, email y token unicos, password_hash, status), person_certificates (los .p12 en MinIO, con is_default), email_verification_codes, password_reset_codes.

5.7.2. Organizacion, unidades y puestos

unit_types, units, relation_unit_types, unit_relations, cargos, unit_positions, position_assignment
Dos garantias elegantes por columna generada:

- unit_positions.head_flag garantiza **un solo jefe por unidad**.
- position_assignments.current_flag garantiza **un solo ocupante actual por puesto**.

Y una vista, unit_org_levels, que con una CTE recursiva sobre unit_relations calcula el nivel organizativo, la unidad raiz y las unidades de nivel 2 y 3 de cada unidad.

Como subdominio propio esta la **contratacion**: vacancies, applications, offers, contracts, contract_origins (con herencia por tabla) y vacancy_visibility.

5.7.3. Entregables, tareas y documentos

tasks, task_items, task_assignments, task_item_handovers (bitacora de traspaso de responsable), documents (1:1 con task_items), document_versions (con version de tipo DECIMAL(4,1)), document_attachments y document_workflow_observations (observaciones, devoluciones y rechazos).

5.7.4. Chat y notificaciones

chat_conversations (con tipos direct, group, thread, process_thread, unit), chat_conversation_participants, chat_messages, chat_message_attachments, chat_message_reads y chat_notifications.

Decision de diseno explicita. En el dominio de chat, person_id, process_id y unit_id son **claves foraneas logicas sin constraint**, “para no acoplar”. Y last_message_id tampoco lleva FK, para evitar un ciclo de dependencias entre tablas.

5.7.5. Auditoria

No existe una tabla de auditoria general (nada de audit_log o bitacora). La trazabilidad es **especifica por dominio**: task_item_handovers para los traspasos, document_workflow_observations para devoluciones, role_assignments.revoked_at y revoked_reason para revocaciones, signature_batch_jobs para el resultado por fichero, y created_at / updated_at con trigger set_updated_at() en la mayoria de tablas.

Capítulo 6

El frontend

Vue 3 + Vite 8 + vue-router 5 + Tailwind v4 + Vitest, sin TypeScript. 216 ficheros en `src/`: 126 `.vue`, 87 `.js` y 3 `.css`.

6.1. Organización: modular por dominio

```
src/
+-- core/      infraestructura transversal (router, httpClient, apiConfig, accessControl)
+-- layouts/   esqueletos de pagina (AppWorkspaceShell, headers, menus)
+-- modules/   LOS DOMINIOS
| +-- admin/   (~80 ficheros) panel de administracion + grafos
| +-- home/    (26)          bandeja de tareas y entregables
| +-- perfil/  (28)          el dossier del usuario
| +-- firmas/  (8)           firma de PDFs
| +-- auth/    (8)           login, registro, bootstrap
| +-- dossier/ (1)           solo el servicio
| +-- procesos/ (1)
+-- shared/    componentes y utilidades reutilizables
```

Lo que NO existe. No hay carpeta `src/assets/` (los estaticos viven en `frontend/public/`), ni `src/views/` ni `src/components/` de nivel raiz: todo esta bajo `modules/` y `shared/`. Y el **chat no es un modulo**: vive como widget compartido en `shared/components/widgets/WorkspaceChatLauncher.vue`.

6.2. El arranque

`src/main.js` tiene 569 bytes y hace lo minimo: crea la app, registra globalmente un unico componente (`font-awesome-icon`), importa los dos CSS globales, marca el entorno local en un atributo del DOM, y monta el router.

`App.vue` solo tiene dos cosas: un `<router-view :key="route.fullPath">` y un `SessionExpiryModal` global gobernado por `useSessionMonitor`.

Sobre Vite: no hay `server.proxy` — el proxy es nginx, externo. El alias `@` apunta a `src`. Y de Tailwind v4 **no hay `tailwind.config.js` ni `postcss.config.js`**: se usa el plugin de Vite mas directivas dentro del propio CSS (`@import "tailwindcss"`; en `shared/styles/tailwind.css`).

6.3. Estado: no hay Pinia ni Vuex

Esto sorprende viniendo de cualquier tutorial de Vue. El patron real es de tres capas:

1. **Composables que poseen su propio estado.** Una funcion `useXxx()` declara sus `ref` y los devuelve. Ejemplo: `useFlowBuilder.js` tiene seis `ref` propios y solo recibe tres dependencias inyectadas. Los comentarios documentan una migracion deliberada desde el antipatron “recibir refs ajenos y escribirlos”.
2. **La sesion vive en localStorage:** la clave `token` (el JWT) y `user` (el usuario serializado como JSON). Las escribe `AuthService.js` y las limpia `clearAuthData()` de `core/utils/tokenUtils.js`.
3. **Eventos del window como bus informal:** `"dossier-updated"`, `"workspace-chat:context-updated"`, `"workspace-chat:open-process"`.

Consecuencia real que hay que conocer. Como el usuario esta en `localStorage` y no en un store reactivo, **los cambios de sesion no propagan por Vue**. De ahi el truco `<router-view :key="route.fullPath">` en `App.vue`, que fuerza a remontar el componente entero en cada navegacion.

6.4. La capa HTTP

`src/core/services/httpClient.js` es una instancia *propia* de `axios` (no el singleton global; el comentario del fichero lo documenta como correccion de un bug de orden de imports) con **un solo interceptor de request**, que anade `Authorization: Bearer <localStorage.token>` si no viene ya puesta.

No hay interceptor de response. Es decir, **no hay refresh automatico ante un 401**. El refresh es manual y explicito: `useSessionMonitor.js` hace polling cada 60 segundos y, cinco minutos antes de que expire el JWT, abre `SessionExpiryModal.vue`, que llama a `/users/refresh-token` con `withCredentials: true` (para que viaje la cookie `refreshToken`). Si falla, cierra sesion.

Las URLs se construyen en `src/core/config/apiConfig.js`, que expone un objeto `API_ROUTES` con unas 110 entradas (unas son cadenas, otras funciones):

```
const API_PORT = import.meta.env.VITE_API_PORT || "3030";
const RAW_API_BASE_URL = String(import.meta.env.VITE_API_BASE_URL || "").trim();
const API_BASE_URL = RAW_API_BASE_URL || `${protocol}://${hostname}:${API_PORT}`;
export const API_PREFIX = `${NORMALIZED_API_BASE_URL}/deasy/v1`; // -> "/api/deasy/v1"
```

Regla documentada en el propio fichero: “*en frontend/src no se importa axios a pelo*”. Y no se fija `baseUrl` a proposito: todas las llamadas pasan la URL completa construida desde `apiConfig`.

Hay 17 ficheros de servicio, todos clases con metodos asincronos sobre `httpClient`. El mas grande es `core/services/ProcessDefinitionPanelService.js` (16,5 KB).

6.5. El router y sus guards

Todo en **un solo fichero**: `src/core/router/index.js`. Doce rutas de primer nivel mas ocho hijas bajo `/perfil`. **Sin lazy-loading**: todas las vistas se importan estaticamente (el `import()` diferido solo se usa para los dos grafos de admin).

El `router.beforeEach` hace, en este orden:

1. Consulta el estado de bootstrap del sistema (¿está instalado?). Si el modo no es **normal**, limpia la sesion y fuerza `/setup`. Si la llamada falla, avisa por consola y **deja pasar** (que el backend este caido no bloquea el enrutado).
2. Si no hay token valido y la ruta es privada, limpia la sesion y va a `/`.
3. Si el usuario es administrador y la ruta tiene `meta.blockedForAdmin`, redirige a `/admin`.
4. Comprueba `meta.requiresProcessManagementAccess` y `meta.requiresAdminAccess`.

Por que se usa meta y no nombres de ruta. Porque vue-router hereda el **meta** a las rutas hijas. `/perfil` tiene ocho hijas (`formacion`, `experiencia`, `referencias`, `capacitacion`, `certificacion`, `investigacion`, `certificados-firma...`). Marcando solo el padre con `blockedForAdmin: true`, las ocho quedan protegidas automaticamente. Si se comprobara por nombre de ruta, habria que acordarse de anadir cada hija nueva a una lista.

Las rutas publicas son `/`, `/register`, `/recover-password`, `/terminos` y `/setup`. La ruta `/logout` no tiene componente: solo un `beforeEnter` que llama al backend, limpia y devuelve `/`.

Coste a tener en cuenta. El guard hace una peticion HTTP (el estado de bootstrap) antes de cada navegacion, sin cache.

`src/core/utils/accessControl.js` es un **espejo declarado** de `RbacService.js` del backend: `hasAnyRole`, `hasPermission` (donde `AdminSistema` pasa todo), `canAccessResource(resource, action)` con respaldo a `.manage`, y un mapa de unas 50 tablas a su recurso RBAC.

El RBAC del frontend es solo cosmetico. Sirve para *ocultar botones* y no enseñar opciones que van a fallar. La autorizacion real la hace **siempre** el backend. Nunca confies en un control de acceso que corre en el navegador del usuario: es codigo que el usuario controla.

6.6. Composables destacados

El patron comun: una funcion `useXxx({ deps })` que **declara y devuelve sus propios ref**, con dependencias inyectadas de solo lectura. Nada de `provide/inject`, nada de singletons reactivos.

`useFlowBuilder.js` (modulo `home`) es el constructor del *flujo routed*: quien elabora (`flowEntrega`) y quien firma (`flowFirma`, con pasos ordenados que llevan `signers`, `approval_mode` y `required_min`). Fue extraido de `HomeView.vue` durante la fase B del refactor del God Object.

Otros: `useProcessPanels.js`, `useDeliverableView.js` (41 KB, el mayor), `useDocumentCenter.js`, `useGeneralTask.js`, `useRecipientSearch.js`, `useWorkspaceChrome.js` (estado del *chrome* compartido por las cuatro vistas que montan `AppWorkspaceShell`), y 24 composables en el modulo `admin` repartidos en seis subcarpetas (`data/`, `fk/`, `forms/`, `modals/`, `processes/`, `ui/`).

6.7. Tiempo real en el cliente

Un unico cliente: `src/core/services/realtimeClient.js` (clase `RealtimeClient` exportada como singleton).

- Autentica con `auth: { token: localStorage.token }` — el mismo JWT que el `httpClient`.

- Resuelve el destino segun la base: si es absoluta, `io(base, {path: "/socket.io"})`; si es relativa (`/api`, detras de `nginx`), `io({path: "/api/socket.io"})` sobre el mismo origen.
- Reconexion infinita con backoff de 1 a 8 segundos; `connect()` es idempotente y **reconecta si el token cambio**.

Su unico consumidor es `WorkspaceChatLauncher.vue`, que escucha `chat.message.created` y `chat.notification.created`.

Codigo muerto detectado. `subscribeProcess` y `unsubscribeProcess` estan definidos en el cliente pero **no tienen ningun llamador** en `src/`.

6.8. Vue Flow y dagre: los dos grafos

Se usan **solo** en el modulo `admin`, para dos grafos interactivos:

- **Organigrama de unidades** — `modules/admin/components/units/UnitGraphView.vue` (57 KB), con nodos y aristas propios (`UnitNode.vue`, `UnitEdge.vue`).
- **Mapa de procesos** — `ProcessGraphView.vue` (55 KB), con `ProcessNode.vue`, `ProcessConfigNode.vue` y `ProcessTemplateNode.vue`.

`dagre` calcula el layout jerarquico de arriba a abajo automaticamente, asi no hay que posicionar nodos a mano. Ambos se cargan con `defineAsyncComponent(() => import(...))`, de modo que Vue Flow y `dagre` **quedan fuera del bundle inicial** y solo se descargan al abrir esas dos pantallas.

Se alcanzan por URL: `/admin/academia/unidades/organigrama` y `/admin/gestiones/procesos/mapa`.

Nota. El constructor de flujos de firma **no** usa Vue Flow: es un formulario de listas (`useFlowBuilder.js`).

6.9. Los “God Objects” (deuda tecnica visible)

Fichero	Tamano
<code>modules/home/views/HomeView.vue</code>	236 KB (unas 5.600 lineas)
<code>modules/admin/components/tables/AdminTableManager.vue</code>	158 KB
<code>modules/firmas/components/FirmarPdf.vue</code>	120 KB
<code>modules/admin/components/modals/AdminDraftArtifactModal.vue</code>	77 KB
<code>modules/admin/components/units/UnitGraphView.vue</code>	58 KB
<code>modules/admin/components/units/ProcessGraphView.vue</code>	56 KB
<code>modules/firmas/components/MultiSignerPanel.vue</code>	52 KB
<code>modules/admin/views/AdminView.vue</code>	46 KB
<code>modules/home/composables/useDeliverableView.js</code>	41 KB

Estan en el plan de calidad y se estan troceando extrayendo composables — asi nacio `useFlowBuilder.js`.

Que es un “God Object”. Un objeto (aqui, un componente) que sabe y hace demasiado: acumula responsabilidades de varios dominios hasta que nadie puede cambiarlo sin miedo. El sintoma es el tamano, pero el problema real es que *cualquier* cambio lo toca, asi que todos los

cambios entran en conflicto y ningun test lo cubre entero. La cura no es partirlo por lineas, sino **extraer responsabilidades completas** (un composable con su estado propio) una a una.

Capítulo 7

El microservicio signer

Es la pieza mas especializada del sistema: Python 3.14 mas **pyHanko**, la biblioteca de firma de PDF.

7.1. Arquitectura interna

signer/app.py (1.545 lineas) arranca **dos cosas a la vez**:

```
rabbit_thread = threading.Thread(target=start_rabbit_worker, daemon=True)
rabbit_thread.start()      # <- el camino REAL de produccion
start_http_server()        # <- solo para depurar (POST /sign, /validate, GET /health)
```

El worker de RabbitMQ escucha dos colas (`deasy.sign.request` y `deasy.sign.validate.request`) con `prefetch_count=1`: **un trabajo a la vez**, para no saturar la memoria con PDFs grandes. Ante cualquier excepcion, registra el error, duerme 5 segundos y reintenta la conexion en un bucle infinito.

Un detalle de higiene: la URL de conexion pasa por `redact_amqp_url()` antes de aparecer en los logs, sustituyendo la contrasena por asteriscos.

7.2. El flujo completo de una firma, paso a paso

1. El frontend llama a `/api/deasy/v1/sign/...`
2. El backend genera un `correlationId`, crea la cola de respuesta `deasy.sign.response.<uuid>`, publica el trabajo y entra en polling.
3. El signer recibe el mensaje y **valida el payload**: exige `signType`, `minioPdfPath`, `stampText`, `finalPath`, `minioCertPath` y `certPassword`; `signType` debe ser `coordinates` o `token`.
4. Crea un directorio temporal con permisos 0700 **fuera de /tmp** — porque ahi se desempaqueta el certificado `.p12` descifrado.
5. Descarga de MinIO el PDF y el certificado.
6. Abre el `.p12` y **rechaza el trabajo si el certificado esta expirado o aun no es valido**, con mensajes explicitos en espanol.
7. Si el PDF **ya tiene firmas embebidas**, salta el paso de limpieza con Ghostscript — aplanarlo destruiria las firmas existentes.

8. Resuelve donde firmar: coordenadas explicitas, o busca un **token de texto** dentro del PDF con `pdfplumber` (`find_marker.py` recorre las palabras de cada pagina y devuelve sus posiciones).
9. **Llama a Node** para generar la imagen del sello (ver seccion siguiente).
10. Por cada coordenada, `pyHanko` crea un campo `sig_<uuid>` e inserta la firma: PAdES, SHA-256, con sellado de tiempo opcional. Con varias firmas, **cada iteracion parte del PDF firmado anterior** (firmas encadenadas).
11. Valida la firma resultante contra el contexto de confianza construido desde `signer/trust/`, y aborta salvo tolerancia explicita en el payload.
12. Sube el PDF firmado a MinIO, reemplazando `minioPdfPath`, y tambien a `finalPath` si es distinto.
13. Publica el resultado en la cola de respuesta; el backend, que estaba haciendo polling, lo recoge.
14. Al salir del bloque `with`, **todo el material temporal se borra**, incluido el `.p12`.

7.3. `sigmaker/`: por que hay Node dentro de un servicio Python

Porque la imagen del sello (QR + nombre del firmante + logo de la PUCE + fecha) se genera con las bibliotecas `canvas` y `qrcode` de Node, mejores para eso que las alternativas de Python. Python lo invoca como subprocesso:

```
subprocess.run(["node", SIGMAKER_DIR + "/cli.mjs",
               stamp_png, stamp_text, final_path, logo_path], check=True)
```

El QR codifica la **URL publica del documento firmado**, para que quien reciba el PDF impreso pueda verificarlo escaneandolo. La fecha se calcula en la zona horaria `America/Guayaquil` y el PNG se escribe con fondo transparente.

Por eso el `docker/signer/Dockerfile` instala `nodejs` y `npm` dentro de una imagen Python, y hace un `npm install` separado en `/opt/sigmaker`.

7.4. `trust/`: la cadena de confianza

```
trust/roots/  <- CAs raiz acreditadas en Ecuador
               firmasegura-root-ca-1.pem
               icert-ec-root.pem
               security-data-root-ca-2.pem

trust/extra/  <- sus subordinadas (se cargan como other_certs, NO como anclas)
               firmasegura-subca-1.pem
               icert-ec-subordinate.pem
               security-data-subca-2.pem
```

El contexto de validacion se construye con `allow_fetching=True` y `revocation_mode="soft-fail"`.

Por que “raiz” y “subordinada” van en sitios distintos. Una *ancla de confianza* (trust anchor) es un certificado que decides crear sin demostracion: es el final de la cadena. Las subordinadas, en cambio, son eslabones intermedios: hay que *poder verificarlas* contra una raiz, no confiar en ellas por decreto. Si metieras una subordinada como ancla, aceptarias como valida una cadena que en realidad no llega a ninguna raiz acreditada.

7.5. Los OIDs ecuatorianos: 500 líneas de app.py

Casi un tercio del fichero está dedicado a **extraer la cedula y el nombre del firmante** de los certificados, porque **cada emisor ecuatoriano usa OIDs propietarios distintos**:

Emisor	Prefijo de OID
Security Data	1.3.6.1.4.1.37746.3.*
ICERT	1.3.6.1.4.1.43745.1.3.*
Firmasegura	1.3.6.1.4.1.61305.3.*

No hay estandar comun: hay que conocer los tres. Además se leen por **dos vías distintas** (`cryptography` y `asn1crypto` sobre el TBS crudo) porque ninguna cubre todos los casos.

7.6. Pruebas del signer

Todas en `signer/tests/`, con `unittest` de la biblioteca estandar (no hay `pytest` en la imagen). **266 métodos test_** repartidos en nueve ficheros:

Fichero	Casos	Que fija
<code>test_certificate_parsing.py</code>	72	DN, extensiones, cedula: quien firmo
<code>test_signing_flow.py</code>	36	Caja del sello, offset del modo token, firmas encadenadas
<code>test_payload_validation.py</code>	34	Orden de guards y mensajes de error
<code>test_pdf_metadata.py</code>	33	Lectura del diccionario /Sig, fechas
<code>test_transport.py</code>	24	Servidor HTTP y publicacion por RabbitMQ
<code>test_validation_report.py</code>	21	Forma exacta del informe que consume el frontend
<code>test_plumbing.py</code>	20	MinIO, material de confianza, Ghostscript, temporales
<code>test_certificate_extensions.py</code>	17	Las dos fuentes de extensiones, con certificados reales
<code>test_certificate_loading.py</code>	9	Carga del PKCS#12 y guards de vigencia

Como apoyo hay `tests/dobles.py` (objetos “pato” que solo exponen lo que `app.py` consulta) y `tests/herramientas.py`, que **no son pruebas**: miden complejidad ciclomatica y anidamiento via AST.

Los conteos no cuadran entre si. `signer/README.md` dice 224 casos, `requirements-dev.txt` y `sonar-project.properties` dicen 229, y el conteo real hoy es 266. Los documentos van por detras del codigo.

Capítulo 8

Infraestructura y despliegue

8.1. Docker: base mas overlays

En vez de un `docker-compose.yml` por entorno (duplicando todo), hay **un fichero base y un overlay** que lo modifica:

```
dev      = compose.base.yml + compose.proxy.yml + compose.dev.yml
qa       = compose.base.yml + compose.qa.yml
prod     = compose.base.yml + compose.prod.yml
ingress  = compose.ingress.yml           (stack independiente)
ingress-bootstrap = compose.ingress.bootstrap.yml (solo HTTP, para el desafio ACME)
```

Que es un “overlay” de compose. Docker Compose permite pasar varios `-f`. Los ficheros se **fusionan** en orden: lo que define el segundo pisa o completa lo del primero. Asi, el base declara los servicios y sus variables comunes, y cada overlay solo dice lo que cambia (puertos, si se construye la imagen o se descarga, montajes de codigo, politicas de reinicio). Evita mantener cuatro ficheros casi identicos que se desincronizan.

Consecuencia concreta: `qa` y `prod` **no cargan** `compose.proxy.yml`, asi que no tienen `nginx-proxy`; su entrada publica es el stack `ingress`, separado y compartido.

8.1.1. Diferencias por entorno

	dev	qa / prod
Imágenes de la app	se construyen localmente (<code>Dockerfile.dev</code>)	se descargan de GHCR con <code>pull_policy: always</code>
Código	bind mount (editas y se recarga)	dentro de la imagen
Proxy	nginx-proxy en el propio stack	un stack ingress separado
Puertos del proxy	8088 / 8443	el ingress publica 80 / 443
PostgreSQL	5432	15432 (qa) / 25432 (prod)
MinIO	9000 / 9001	9100 / 9101 y 9200 / 9201
Buckets	deasy-*	deasy-qa-* y deasy-prod-*
PAdES-LTA	desactivado	activado en prod
Endurecimiento	—	no-new-privileges , read_only en frontend, tmpfs , healthchecks propios

RabbitMQ solo se publica en dev, y atado a 127.0.0.1. El comentario del compose explica el motivo: la interfaz de gestion de RabbitMQ **muestra el cuerpo de los mensajes**, y ahi viaja `certPassword`, la contraseña del certificado de firma. En `qa` y `prod` los puertos no se publican en absoluto.

8.1.2. Los servicios del fichero base

Servicio	Imagen	Notas
postgres	postgres:17	healthcheck <code>pg_isready</code> ; unico motor de datos
rabbitmq	rabbitmq:3-management	healthcheck <code>rabbitmq-diagnostics ping</code>
minio	minio/minio:...	<code>server /data -console-address ":9001"</code>
minio-bootstrap	minio/mc:...	perfil <code>storage-init</code> ; crea los buckets y hace mc mirror de las semillas
backend	sin imagen en base	la define cada overlay
frontend	sin imagen en base	idem
signer	sin imagen en base	<code>SIGNER_PORT=4000</code> , <code>SIGMAKER_DIR=/opt/sigmaker</code>
analytics	sin imagen en base	perfil <code>workers</code> ; hoy es un marcador de posicion

8.2. scripts/docker-env.sh: la interfaz comun

Es la **unica forma correcta** de tocar el stack. Todo lo que va despues del entorno se reenvia literalmente a `docker compose`:

```
bash scripts/docker-env.sh dev up -d --build
bash scripts/docker-env.sh dev logs -f backend
bash scripts/docker-env.sh dev ps
bash scripts/docker-env.sh dev exec -T backend npm run test:unit
bash scripts/docker-env.sh dev down
```

Resuelve el fichero `.env` correcto (con `DEASY_ENV_FILE` como gancho para apuntar al `.runtime` en despliegues), valida que existan todos los ficheros antes de invocar nada, exporta `DEASY_CONTAINER_ENV_FILE` y hasta convierte rutas si detecta `cygpath` en Windows.

Regla del proyecto. *“Builds y tests deben correr dentro de los contenedores via `scripts/docker-env.sh`, no con `npm` o `npx` en el host”.* Y al **añadir dependencias al frontend** hay que instalar **dentro del contenedor**: el volumen de `node_modules` sombrea el de la imagen, asi que un `pnpm install` en el host **no se ve** dentro.

8.2.1. Los demas scripts operativos

Script	Que hace
<code>apply-env.sh</code>	Motor comun de despliegue. Aplica el tag de imagen, crea la red externa si falta, limpia contenedores antiguos que retengan los puertos 80/443, y levanta el stack. Soporta <code>DEASY_DRY_RUN=1</code> .
<code>deploy-env.sh</code>	Envoltorio fino que llama a <code>apply-env.sh</code> en modo <code>gh-actions</code> . Es lo que ejecuta el workflow por SSH.
<code>server-pull-deploy.sh</code>	Modo <i>server-pull</i> : exige rama correcta y arbol limpio, hace <code>git pull -ff-only</code> y despliega.
<code>_backend_db_exec.sh</code>	Libreria compartida. Para produccion exige un fichero de aprobacion dentro del repo e ignorado por git .
<code>reset-db.sh</code>	Resetea solo la base de datos dentro del contenedor.
<code>reset-system.sh</code>	Reset completo a instalacion virgen: base + almacenamiento, recicla backend y signer, y recarga nginx (porque el proxy de dev cachea la IP del backend).
<code>rabbitmq-migrar-usuario.sh</code>	Crea el usuario real de RabbitMQ y despues borra guest . Idempotente.
<code>bootstrap-ingress-cert.sh</code>	Emite el certificado publico con certbot (ver mas abajo).
<code>signer-coverage.sh</code>	Genera el informe de cobertura de Python para Sonar, reescribiendo las rutas para que las resuelva.

8.3. nginx: dos configuraciones distintas

8.3.1. Mecanica de plantillas

No existe un directorio `nginx/conf.d/` en el repositorio. Los tres directorios `*-conf.d/` se montan como `/etc/nginx/templates`, y el entrypoint oficial de la imagen `nginx` aplica `envsubst` a cada `*.template` generando los `.conf` finales. De ahí que las plantillas usen variables `${VAR}` literales.

8.3.2. Desarrollo

`nginx/app-conf.d/default.conf.template`: un `server_name _` que no discrimina.

```
location = /api { return 301 /api; }
location /api/ { proxy_pass http://${BACKEND_UPSTREAM}/; }
location /      { proxy_pass http://${FRONTEND_UPSTREAM}; }
```

La barra final del proxy_pass. Es la diferencia entre que funcione y que no. Con `proxy_pass http://backend:3030/;` (con barra), `nginx` **sustituye** la parte que casó (`/api/`) por la barra, así que `/api/deasy/v1/users` llega al backend como `/deasy/v1/users`. Sin la barra final, la ruta se pasaría íntegra y el backend recibiría `/api/deasy/v1/users`, que no tiene ninguna ruta registrada: 404.

El puerto 80 sirve solo tres cosas: `/healthz`, el desafío ACME (`/.well-known/acme-challenge/`) y una redirección 301 a HTTPS.

8.3.3. Producción y QA

`nginx/ingress-conf.d/default.conf.template` es un **ingress compartido** que reparte por cabecera `Host`: `fresvel.com` y `www.fresvel.com` van a prod, `qa.fresvel.com` va a qa. Un cuarto bloque `default_server` devuelve **404** a cualquier `Host` no reconocido. Aquí la reescritura se hace con `rewrite ^/api/(.*)$ /$1 break;`.

Detalle importante: los upstreams se asignan a **variables** junto con `resolver 127.0.0.11 ipv6=off valid=10s`. Eso fuerza a `nginx` a **re-resolver el DNS de Docker cada 10 segundos** en vez de cachear la IP al arrancar. Es justo el problema que sufre el proxy de dev (resolución estática), y por eso `reset-system.sh` tiene que recargarlo tras reciclar el backend.

8.3.4. Rutas efectivas

`nginx` solo conoce dos prefijos: `/api/` y `/`. Todo lo demás son rutas internas de Express:

```
https://host/api/deasy/v1/users -> nginx quita /api -> backend /deasy/v1/users
https://host/api/deasy/docs     -> nginx quita /api -> backend /deasy/docs   (Swagger UI)
https://host/api/socket.io      -> nginx quita /api -> backend /socket.io    (WebSocket)
```

El upgrade a WebSocket funciona gracias a las cabeceras `Upgrade` y `Connection` y al `map $http_upgrade $connection_upgrade` definido en `nginx/nginx.conf`.

8.3.5. Certificados y ACME

- `nginx/certs/dev/` y `nginx/certs/qa/`: **autofirmados versionados** en git (generables con `nginx/scripts/generate-self-signed.sh`).

- `nginx/certs/public/`: **no versionado**; ahí deja `scripts/bootstrap-ingress-cert.sh` el certificado de Let's Encrypt.
- `nginx/acme/`: el *webroot* del desafío HTTP-01; montado de solo lectura en proxy e ingress, y de **lectura-escritura** en el bootstrap, para que certbot pueda escribir el fichero de reto.

Por que existe un “ingress-bootstrap”. Let's Encrypt verifica que controlas el dominio pidiéndote colocar un fichero en `http://tudominio/.well-known/acme-challenge/xxx`. Pero el ingress normal solo escucha en HTTPS con un certificado... que aun no tienes. Es el problema del huevo y la gallina. La solución: un nginx mínimo, **solo HTTP**, que sirve el reto; se emite el certificado, se copia, se baja el bootstrap y se levanta el ingress definitivo.

8.4. CI/CD

8.4.1. `cd-multienv.yml`: ocho jobs

```
prepare (rama -> entorno)
  |-- frontend-lint      (pnpm run lint)
  |-- backend-checks     (npm run check:imports + npm run test:unit)
  +-- compose-validate   (docker-env.sh <env> config, para los 4 entornos)
      |
      v
  publish-images         (matriz de 4 imagenes -> GHCR)
      |                  tags: :dev|:qa|:prod y :sha-XXXXXXX
      v
  deploy-app-push / deploy-app-manual / deploy-ingress
    (rsync de docker/ y nginx/ + SSH, solo si DEPLOY_DELIVERY_MODE == 'gh-actions')
```

Mapecto de ramas: `develop` → `dev` (publica imagenes, **no despliega**); `qa` → `qa`; `main` → `prod`. El trabajo se hace en `develop`; `main` es la rama de producción.

El `rsync` solo sube `docker/`, `nginx/` (excluyendo `certs/`) y cuatro scripts. El fichero de variables reales llega como secreto de GitHub y se vuelca en `docker/.env.<env>.runtime` en el servidor.

8.4.2. Dos modos de entrega

- **Push**: GitHub Actions entra por SSH y despliega.
- **Pull**: unidades `systemd` en el servidor (`deploy/systemd/deasy-server-pull@.timer`) que cada **15 minutos** hacen `git pull` y redesplicgan (`OnBootSec=5m`, `OnUnitActiveSec=15m`, `Persistent=true`).

El segundo modo existe para servidores **sin IP pública estable**, donde GitHub no puede iniciar la conexión.

8.4.3. `sonar.yml`: solo manual, y en verde si falta configuración

Se dispara solo con `workflow_dispatch`. Un primer paso *guard* comprueba si existen los secretos `SONAR_HOST_URL` y `SONAR_TOKEN`; si faltan, escribe una explicación en el resumen del workflow y **termina en verde**.

La razón documentada. El SonarQube del proyecto está autoalojado en `localhost:9002` y **un runner de GitHub no puede alcanzarlo**. Poner el análisis en el workflow de push lo dejaría en rojo para siempre, lo que entrena a todo el mundo a ignorar los fallos de CI. Sonar en CI está

pendiente de una decision de infraestructura (publicarlo con TLS o migrar a SonarCloud), no de codigo.

8.5. SonarQube: como esta montado

Pieza	Donde
Stack	scripts/sonar/compose.yml — SonarQube <i>community</i> + PostgreSQL 16, puerto 9002
Lanzador	scripts/sonar/scan.sh — sonar-scanner-cli dockerizado
Config	sonar-project.properties — projectKey=deasy; fuentes backend, frontend/src, signer, scripts, con los tests declarados aparte
CI	.github/workflows/sonar.yml

Cuatro cosas que cuestan una medicion entera.

1. **Regenera los DOS informes de cobertura antes de escanear.** Si no, Sonar lee los de la corrida anterior **sin quejarse**. Y si las rutas SF: del lcov no son relativas a la raiz del repo, los descarta **en silencio** y la cobertura vuelve a cero.
2. **Al consultar la API, filtra con resolved=false.** Por defecto incluye las cerradas y las *won't fix*, y los conteos salen inflados.
3. **El escaneo se procesa despues de subirse.** Consultar metricas justo al terminar devuelve las viejas: espera a que la tarea diga **SUCCESS**.
4. **No toques sonar.projectVersion.** Mueve el periodo de New Code y tira la serie historica, que es el unico termometro fiable que hay.

Marcar no es arreglar. Los falsos positivos se marcan en Sonar con justificacion; hay varios que **no** hay que “corregir”. Sonar rastrea la incidencia por el **hash de la linea**: un renombrado puro conserva la marca, pero reescribir la linea la pierde.

Capítulo 9

Testing: dos niveles que no se mezclan

Esta es una regla dura del proyecto, y la ubicacion de los ficheros **no es estetica**: `sonar-project.properties` distingue tests de codigo de produccion **por patron**, y un fichero mal colocado o mal nombrado se analiza **como codigo de produccion**. Asi es como las contraseñas de fixture acabaron contando como vulnerabilidades del sistema.

Nivel	Donde	Nombre	Que prueba
Unitario	junto al modulo que prueba	<modulo>.test.js (.test.mjs si el modulo es ESM .mjs)	Una unidad, sin red ni base de datos
Caracterizacion	backend/tests/characterization/flows/	Flows/*.test.mjs	Contrato HTTP extremo a extremo contra <i>goldens</i>

9.1. Reglas de ubicacion

- `*.test.js` y `*.test.mjs` son los **unicos sufijos validos**. No uses `*.spec.js`: no hay ni uno en el repositorio, y anadirlo obliga a mantener un patron muerto en la configuracion de Sonar y en los globs de `test:unit`.
- Un test unitario nuevo tiene que **caer dentro de los globs de backend/package.json** → `test:unit` (`config/`, `services/**`, `utils/**`, `middlewares/**`, `controllers/**`, `errors/**`). Si lo pones fuera, **no lo ejecuta nadie** y no te vas a enterar.
- Si el modulo vive en otra carpeta, **amplia el glob en el mismo commit y en los DOS sitios**: `test:unit` y `test:unit:coverage` llevan la misma lista duplicada (el segundo con prefijo `backend/`, porque corre desde la raiz del repo para que las rutas del lcov le valgan a Sonar). Si solo tocas uno, el test corre pero **no cuenta para la cobertura**.
- No metas tests unitarios en `backend/tests/`: esa carpeta es del harness de caracterizacion.
- Los tests del frontend van junto al componente o composable (vitest los descubre solo).

Una suite que no arranca cuenta como fallo, no como “0 tests”. Vitest marca *Failed Suite* con 0 casos cuando el error ocurre al importar — típicamente un `vi.mock` incompleto de un modulo que ahora tira de otro. Mira la linea **Test Files**, no solo la de **Tests**.

9.2. Los tests unitarios

Backend: 22 ficheros, **361 casos**, con `node -test` — el runner nativo de Node, sin Jest ni Vitest. Los mas nutridos:

Fichero	Casos
<code>config/postgres.dialect.test.js</code>	38
<code>services/documents/FillRequestWorkflowService.test.js</code>	37
<code>services/admin/crud/validation.test.js</code>	33
<code>services/admin/templates/workflows.test.js</code>	28
<code>services/sign/PdfSigningService.test.js</code>	27
<code>services/sign/BatchSigningService.test.js</code>	21
<code>controllers/users/user_controller.primitives.test.js</code>	21
<code>services/tasks/GeneralTaskService.test.js</code>	19

Frontend: 18 ficheros, unos **261 casos** con Vitest y `@vue/test-utils`. La configuracion vive en el bloque `test:` de `vite.config.js` (no hay `vitest.config.js`). El entorno por defecto es `node`; los once tests que montan componentes ponen la pragma `// @vitest-environment jsdom` en la primera linea. **No hay pruebas E2E** (ni Playwright ni Cypress).

9.3. Los characterization tests (golden master)

Que es un golden master. En vez de escribir “espero que devuelva `{x: 1}`”, **grabas** lo que devuelve hoy y lo guardas en un JSON. En cada ejecucion comparas contra el JSON grabado. Si cambia, el test falla y **el diff de git es la evidencia** de lo que cambiaste.

Sirve para **blindar refactors** en codigo que no entiendes del todo: no documentas lo que *deberia* hacer, sino lo que *hace*, rarezas incluidas. Es la red de seguridad ideal cuando heredas un sistema.

En Deasy son **15 suites**, **150 tests** y **256 claves de snapshot**, sin dependencias externas: `fetch` nativo mas `node:test` y `node:assert/strict`.

9.3.1. Anatomia del harness

Pieza	Que hace
<code>config.mjs</code>	Todo parametrizable por entorno: <code>API_PREFIX</code> , <code>BASE_URL</code> , <code>timeouts</code> , <code>SNAPSHOT_MODE</code> (<code>compare</code> o <code>update</code>), los tres usuarios de referencia y la <code>FIXTURE</code> con los IDs esperados.
<code>lib/http.mjs</code>	<code>request/get/post/put/patch/del</code> ; devuelve siempre <code>{status, ok, headers, body}</code> y nunca lanza por codigo de estado .
<code>lib/auth.mjs</code>	<code>login(user)</code> y <code>tokenFor(userKey)</code> con cache por ejecucion.
<code>lib/snapshot.mjs</code>	El motor: <code>matchSnapshot(suite, key, actual)</code> . En modo <code>update</code> escribe y pasa; en <code>compare</code> falla, y tambien falla si la clave no existe .
<code>lib/normalize.mjs</code>	Enmascara lo volatil (<code>timestamps</code> , <code>iat</code> , <code>exp</code> , <code>tokens</code>) con <code>"<normalized>"</code> . Los IDs se conservan a proposito.
<code>lib/readiness.mjs</code>	Polling cada 2 segundos hasta que el sistema responde.
<code>lib/db.mjs</code>	Excepcion declarada: pool de PostgreSQL directo solo para limpieza . La regla escrita es <i>“se usa para LIMPIAR, nunca para ASERTAR”</i> .
<code>setup/bootstrap_system.mjs</code>	Construye la fixture con el bootstrap real por HTTP , no con un seed SQL paralelo (que era la fuente de verdad anterior y derivaba). Idempotente, y falla si los IDs divergen.
<code>setup/seed_execution.mjs</code>	Simbra datos de ejecucion via API.
<code>__snapshots__/</code>	15 JSON, uno por suite, con claves ordenadas alfabeticamente e indentacion 2, para que el diff de git sea legible .

9.3.2. El ciclo de trabajo

```
test:char:fixture = reset.mjs db --yes && test:char:bootstrap && test:char:seed
test:char:run     = test:char:fixture && test:char
```

```
test:char:capture = test:char:fixture && SNAPSHOT_MODE=update npm run test:char
test:char         = node --test --test-concurrency=1 tests/characterization/flows/*.test.mjs
```

Los prefijos **zz**, **zzz** y **zzzz** en los nombres de fichero fuerzan el orden alfabético de ejecución (el runner corre con `-test-concurrency=1`), poniendo al final las suites que *escriben* datos: `zz_task_generation`, `zz_template_lifecycle`, `zzz_artifact_draft`, `zzzz_sign_batch`, `zzzz_sign_workflow`.

test:char:run RESETEA la base de dev. Hace reset, bootstrap y seed. Es lo normal para caracterización, pero **no lo lances si tienes datos que quieras conservar**. Para actualizar los goldens: `test:char:capture`.

La regla de oro. Un refactor no cambia un golden. Si un snapshot se mueve durante un refactor puro, o rompiste algo o el test estaba mal. En un *fix*, el diff del golden es la prueba del arreglo.

9.4. Comandos de validación por módulo

```
# Frontend: lint + tests unitarios, y además verificar en el navegador
bash scripts/docker-env.sh dev exec -T frontend pnpm run lint
bash scripts/docker-env.sh dev exec -T frontend pnpm run test:unit
bash scripts/docker-env.sh dev exec -T frontend pnpm run test:unit:coverage
bash scripts/docker-env.sh dev exec -T frontend pnpm run build

# Backend: no tiene lint, pero SI tiene tests
bash scripts/docker-env.sh dev exec -T backend npm run test:unit
bash scripts/docker-env.sh dev exec -T backend npm run test:char:run
bash scripts/docker-env.sh dev exec -T backend npm run check:imports # OBLIGATORIO tras mover
                                código
bash scripts/docker-env.sh dev exec -T backend npm run test:unit:coverage
```

9.5. Las reglas al mover código

Del CLAUDE.md, porque cada una nació de un incidente real:

1. **Refactor = mover código, NO reescribir comportamiento.** Si cambias que hace algo, no es un refactor.
2. **node -check valida SINTAXIS, no imports.** Por eso `check:imports` es obligatorio, y comprobar que el backend arranca **no** lo sustituye.
3. **No injertes casos especiales en el camino genérico.**
4. **Extrae por script, no a mano,** y verifica `count == 1` antes de borrar cada bloque.
5. **El SQL no lo valida nadie** hasta que se ejecuta esa rama. Pruebalo con `PREPARE` en `psql`.

Capítulo 10

Cosas que te van a confundir (y no son bugs)

Esta lista ahorra horas. Son incoherencias reales del repositorio, verificadas.

1. **backend/README.md esta obsoleto.** Habla de MongoDB (URI_MONGO), MariaDB y puerto 3000. El código usa PostgreSQL y el puerto 3030. **El CLAUDE.md es la fuente fiable**, no los README.
2. **El README.md raiz enlaza documentos que ya no existen** (docs/02-dominio-datos/modelo-datos.md, MER_SQL.sql, docs/01-arquitectura/). Están en docs/docs-md-antiguos/. Lo único que hay hoy en docs/02-dominio-datos/ es consolidado.dbml (31 KB, con unas 205 relaciones).
3. **amqplib esta en package.json pero no se importa en ningún sitio.** Es una dependencia muerta: la integración real con RabbitMQ es por su API HTTP de gestión.
4. **sqlTables.js declara campos de template_artifacts que ya no son columnas de esa tabla** (template_code, display_name, description, template_scope, template_seed_id, owner_person_id). **No es un bug:** SqlAdminService los rutea por JOIN a deliverables, exponiéndolos con los mismos nombres (d.code AS template_code). Es una fachada deliberada.
5. **“Entregable” significa dos cosas** según el contexto: task_items (la instancia) y deliverables (la identidad de la plantilla).
6. **Las fases del plan de calidad se llaman A, B, C... por el orden en que se descubrieron**, no por prioridad ni por tema.
7. **test:char:run** resetea la base de dev.
8. **rounded-lg vale 16px** en este frontend, no lo que dice la documentación de Tailwind: theme.css pisa el espacio de nombres de radios de Tailwind. La escala está efectivamente invertida.
9. **La contraseña del gestor NO es Demo1234!, es Gestor1234!.**
10. **No existe artifact_stage** pese a estar descrito en la documentación de arquitectura. La realidad es template_artifacts.lifecycle_state con tres valores.

11. **No hay tabla de auditoria transversal.** Solo bitacoras especificas por dominio.
12. **docker/docs/Dockerfile es una imagen huérfana:** no la referencia ningun compose ni el workflow.
13. **docker/README.md documenta puertos de RabbitMQ en qa y prod que los overlays ya no publican.**
14. **program_router.js y unit_router.js son literalmente el mismo router duplicado,** y **notification_router.js** duplica dos endpoints de **chat_router.js**.
15. **backend/public/ esta vacia** pese a montarse con **express.static**.
16. **Coexisten dos configuraciones de ESLint en el frontend:** **eslint.config.cjs** (flat config, la efectiva con ESLint 10) y **.eslintrc.js** (legacy, residual e inerte).
17. **No hay .env dentro de frontend/ ni de backend/:** viven en **docker/**, y el modelo de referencia es **docker/.env_model**.

10.1. Usuarios y accesos de referencia

Los crea el **bootstrap** (/setup → “usar datos de ejemplo”). **No hay ningun seed SQL alternativo.**

Perfil	Cedula	Contraseña	Notas
admin	1234567890	Demo1234!	bloqueado en el espacio de usuario
gestor	0987654321	Gestor1234!	de momento tiene tambien rol de usuario
usuario	1122334455	Demo1234!	

El admin no puede probar el dossier ni las firmas. El router bloquea el espacio de usuario para el administrador con **meta: { blockedForAdmin: true }** (el guard lo redirige a **/admin**): **/home**, **/home/documentos**, **/home/firmas** y **/perfil** con todas sus hijas, porque vue-router hereda el meta. **Para probar el dossier o las firmas hay que entrar como gestor o como usuario.**

Puertos de dev: proxy HTTP **8088** y HTTPS **8443** (la API bajo **/api/deasy/v1**); el puerto directo del backend es **3030**. Credenciales de SonarQube: usuario **admin**, contraseña **Demo1234!Demo** — pero **la API no acepta esa contraseña por basic auth**: todo va por token.

Capítulo 11

Por donde empezar a leer

Ruta de lectura sugerida, de menor a mayor dificultad:

#	Fichero	Por que
1	CLAUDE.md	Las reglas del proyecto, y es lo mas actualizado
2	backend/index.js	El arranque completo en unas 200 lineas
3	backend/config/apiPaths.js	El mapa de URLs de toda la API
4	backend/services/documents/DocumentWorkflow.js	El Restful Objective : una sola responsabilidad, se lee de una sentada
5	backend/config/rbacCatalog.js	Entiendes el modelo de permisos entero
6	backend/database/postgres_schema.sql	Los comentarios explican el porque de cada decision
7	docs/arquitecturas/modelo-emision-entregable.md	El mapa de verdad de item_mode
8	docs/planes/referencia/calidad-y-medicion.md	Que esta mal, que se esta arreglando, y que NO hay que tocar
9	frontend/src/core/router/index.js	El mapa del frontend
10	signer/app.py	Dejalo para el final: es denso

11.1. Los tres documentos maestros

- docs/planes/referencia/calidad-y-medicion.md — el **documento maestro** de deuda tecnica y complejidad: mapa de fases con su estado, linea base de SonarQube, ranking de ficheros y funciones, y la lista de **lo que NO hay que tocar** (sqlTables.js y los falsos positivos entran ahi). **Leelo antes de proponer un refactor.**
- docs/planes/referencia/cobertura.md — la cobertura. Y lo primero que dice: el gate **no pide 80 % global** (eso seria trabajo de anos), pide **80 % de lo nuevo.**
- docs/planes/referencia/patron-es-diseño.md — **cuando usar un patron de diseño y cuando no.** En este repositorio la complejidad se cura con **tablas y extraccion**, no con jerarquias; hay tres sitios donde un patron GoF si se gana el sueldo, y una lista de donde seria sobreingenieria.

11.2. Las tres cosas que mas cuestan

Si algo se te atraviesa, casi seguro es una de estas tres:

1. **El motor de procesos** (serie → regla → flujo, y los tres **item_mode**). Capitulo 5 de este documento.
2. **El flujo de firma completo**, desde el clic hasta el PDF firmado en MinIO, pasando por la cola. Capítulos 4 y 7.

3. **El panel de administracion generico** (`sqlTables.js` + `SqlAdminService.js` + `tableHooks.js`, y su contraparte `AdminTableManager.vue`). Capítulos 4 y 6.